

conference 2

一、余弦相似度

📖 描述与解释

长度独立性：余弦相似度优于其他相似度度量方法，原因在于它不依赖于记录的长度。这意味着无论姓名是长是短，余弦相似度都能公平地衡量两个姓名之间的相似程度，而依赖长度的度量方法则会产生偏差。

归一化等价点积：在方案概述部分，论文详细说明了归一化处理的重要性：当查询向量和数据库向量都完成 L2 归一化后，两个向量的点积就直接等于它们的余弦相似度，无需额外的除法运算。这一特性极大简化了加密域中的相似度计算，因为点积是同态加密原生支持的操作。

聚类与匹配：在聚类部分，论文使用余弦相似度作为聚类的距离度量标准，确保相似的姓名被划分到同一个聚类中，从而有效缩小搜索范围。在匹配阶段，余弦相似度与预设的阈值进行比较。论文中给出的参考阈值为 0.9，表示只有 90% 以上相似的姓名才被认为是真正的匹配。

对应代码

在本模块提供的完整协议代码中，余弦相似度并非通过显式的公式计算，而是通过归一化后的点积运算隐式完成。

```
# 对聚类中心、数据库矩阵、查询向量进行 L2 归一化
centroids_B = centroids_B / np.linalg.norm(centroids_B, axis=1, keepdims=True)
db_matrix_B = db_matrix_B / np.linalg.norm(db_matrix_B, axis=1, keepdims=True)
query_A = query_A / np.linalg.norm(query_A)
```

其中`np.linalg.norm`在不指定`ord`参数时就是计算L2范数的函数

`axis=1`：沿着列方向计算，结果是每一行的范数

`centroids_B` 的每一行是一个聚类中心向量，`db_matrix_B` 的每一行是一个样本向量，所以用 `axis=1` 对每个向量独立归一化。

`keepdims=True`：保持结果形状为列向量，方便后续逐行除法

代码中的 `dot_ct_ct` 函数接收两个加密向量并返回它们的点积，由于输入向量都已经在明文阶段完成归一化，这个点积结果在密文状态下就代表了加密的余弦相似度：

```
enc_exact_cos_score = api.dot_ct_ct(enc_query_A, ct_target_B)
```

对应接口

接口名称	核心作用
dot_ct_ct	两个密文向量的点积，由于输入向量已归一化，结果直接等于加密的余弦相似度

二、点积算法

99 描述与解释

在方案的加密计算部分，点积是同态加密中支持的基本操作。点积由两个步骤组成：先对向量的每个分量执行乘法，再将所有乘积结果累加起来。

密文-明文点积

这是协议中使用最频繁的操作，主要用于两个场景：

- 1. **质心匹配阶段**：响应方收到查询方发来的加密查询向量后，计算该加密向量与所有聚类中心（明文存储）之间的相似度。采用密文-明文点积即可高效完成，无需消耗重线性化资源。
- 2. **列选择阶段**：响应方收到加密的独热编码指示向量后，与列矩阵进行密文-明文点积，从而完全在加密域中高效提取出目标姓名。

密文-密文点积

用于计算查询向量与候选姓名向量之间的余弦相似度。这是整个协议中计算开销最重的操作，因为两个密文相乘会导致密文维度从二维膨胀到三维，必须通过重线性化操作将其降回二维才能继续计算。尽管代价较高，但这一步必不可少，它确保了在不暴露数据的前提下完成相似度计算。

对应代码

代码中的接口类封装了点积运算的两种形式。

密文-明文点积通过 dot_ct_pt 方法实现：

```
@staticmethod
def dot_ct_pt(ct_vec: ts.CKKSVector, pt_vec: np.ndarray) -> ts.CKKSVector:
```

```
return ct_vec.dot(pt_vec.tolist())
```

在协议中，此方法被调用两次（质心匹配与列选择）：

```
# 第一次：质心匹配循环
for i in range(k_clusters):
    score = api.dot_ct_pt(enc_query_A, centroids_B[i])
    enc_sim_with_centroids.append(score)

# 第二次：列选择循环
for col_idx in range(sig_len):
    column_vec = db_matrix_B[:, col_idx]
    extracted_feature = api.dot_ct_pt(enc_one_hot_S, column_vec)
    extracted_target_enc.append(extracted_feature)
```

- 质心匹配循环：对enc_query_A查询样本的编码向量和centroids_B[i]B数据集第i个聚类的中心向量做点积运算
- 得到 k_clusters 个相似度分数，表示查询样本与各个聚类中心的匹配程度，用于后续选择最匹配的聚类。

用一个固定模板（enc_one_hot_S），去扫描db_matrix_B的每一列，计算模板与每一列的相似度，最终得到一组分数，表示模板在各个位置上的匹配程度。

在点积算法中，查询向量指的是**查询方 A 对自己的姓名进行 MinHash 编码、归一化处理后得到的数值向量**，并且这个向量已经被 A 用 CKKS 公钥加密成了密文状态。

在整个协议的不同阶段，查询向量以两种形式存在：

形式	编码长度	用途
$E(N_A)$	200 维	用于第一轮与聚类中心的比较（已标准化）
$E(N_A)$	50 维	用于第二轮与候选姓名的余弦相似度计算（仅归一化）

密文-密文点积通过 dot_ct_ct 方法实现：

```
@staticmethod
def dot_ct_ct(ct_vec1: ts.CKKSVector, ct_vec2: ts.CKKSVector) ->
ts.CKKSVector:
    return ct_vec1.dot(ct_vec2)
```

用于计算最终的加密余弦相似度：

```
enc_exact_cos_score = api.dot_ct_ct(enc_query_A, ct_target_B)
```

重线性化：

重线性化密钥的生成发生在上下文设置阶段：

```
def setup_ckks_context():
    ctx = ts.context(ts.SCHEME_TYPE.CKKS, poly_modulus_degree=8192,
coeff_mod_bit_sizes=[60, 40, 40, 60])
    ctx.global_scale = 2 ** 40
    ctx.generate_galois_keys()
    ctx.generate_relin_keys()
    return ctx
```

重线性化没有显式的调用语句，而是在密文-密文点积时自动发生：

```
@staticmethod
def dot_ct_ct(ct_vec1: ts.CKKSVector, ct_vec2: ts.CKKSVector) ->
ts.CKKSVector:
    return ct_vec1.dot(ct_vec2)
```

当执行 `ct_vec1.dot(ct_vec2)` 时，TenSEAL 库内部会：

1. 对两个密文向量的每个对应分量执行密文乘法
2. 乘法会产生三维密文
3. 自动调用重线性化，使用之前生成的 `relin_keys` 将三维密文降回二维
4. 将所有分量的结果累加起来
5. 返回最终的二维密文

对应接口

接口名称	核心作用
<code>dot_ct_pt</code>	密文与明文向量的点积运算，用于质心匹配和列选择
<code>dot_ct_ct</code>	密文与密文向量的点积运算，用于精确余弦相似度计算

三、阈值判断

📖 论文中的描述与解释

论文明确描述了阈值减法的操作：从加密的余弦相似度中减去阈值。

这个操作将原本需要比较大小的复杂判断问题，转化为了简单的正负号判断问题。

- 若原始相似度大于或等于阈值，减去阈值后的结果大于或等于 0。
- 若原始相似度小于阈值，减去阈值后的结果小于 0。

这样一来，响应方只需要返回加密的差值，查询方解密后查看数值的正负，就能知道是否匹配。整个过程无需任何额外的比较电路，完全利用同态加密的加法性质完成。阈值减法只涉及密文与明文的加法操作，不会引入额外的噪声，计算代价极低。

对应代码

代码中的阈值判断通过 `subtract_threshold` 方法实现：

```
@staticmethod
def subtract_threshold(ct_score: ts.CKKSVector, tau: float) -> ts.CKKSVector:
    return ct_score - tau
```

在代码中，类型重载了减法运算符，使得密文与浮点数的减法操作极其直观。此方法的调用如下：

```
enc_score_sub = api.subtract_threshold(enc_exact_cos_score, tau)
```

减法运算在加密域中完成，响应方无法知道减法后的具体数值，查询方解密后才能看到结果。

对应接口

接口名称	核心作用
<code>subtract_threshold</code>	从加密分数中减去相似度阈值，将大小比较转化为正负号判断

四、随机隐私化

📖 论文中的描述与解释

论文在最后一步明确描述了随机掩码操作：将减去阈值后的分数乘以一个随机数（随机选择的正整数）。

随机掩码带来了三重隐私保护效果：

- 1. **数值混淆**：查询方无法从分数数值的大小反推出原始的相似度值。即使原始相似度是 0.95，减去阈值 0.9 后得到 0.05，再乘以随机数（如 500）后变成 25，查询方完全无法逆推原始数据。
- 2. **跨列隔离**：每一列使用独立生成的随机数，防止查询方在不同列之间进行横向比较推断。
- 3. **符号保持**：随机数总是正数，因此乘法操作保持原始差值的正负号不变，确保了查询方仍然能够正确判断是否匹配，但无法知道具体有多匹配。

对应代码

代码中的随机隐私化通过 `apply_mask` 方法实现：

```
@staticmethod
def apply_mask(ct_score: ts.CKKSVector) -> ts.CKKSVector:
    r_j = random.randint(10, 1000)
    return ct_score * r_j
```

这个方法在内部生成一个 10 到 1000 之间的随机正整数进行密文相乘，在混淆效果与数值稳定性之间取得平衡。在主协议中，它是发送回查询方前的最后一步处理：

```
enc_final_score = api.apply_mask(enc_score_sub)
```

查询方解密后，只能根据数值的正负来判断是否匹配：

```
final_score_pt = enc_final_score.decrypt()[0]
catch = 1 if final_score_pt > 0 else 0
```

对应接口

接口名称	核心作用
<code>apply_mask</code>	对加密分数乘以独立生成的随机正整数，彻底隐藏真实相似度数值

五、完整协议中的调用顺序

三 执行流水线

上述四个操作严格按照以下顺序执行：

1. **初始化与归一化**：生成聚类中心、数据库矩阵和查询向量，并进行 L2 归一化。
2. **加密查询**：将查询向量加密为密文对象。
3. **密文-明文点积（质心匹配）**：循环调用接口计算加密查询与每个聚类中心的相似度。
4. **解密与选择**：查询方解密得到各聚类中心的相似度，选出最匹配的聚类，构造独热向量并加密。
5. **密文-明文点积（列选择）**：循环调用接口从每一列中提取出最匹配聚类对应的姓名。
6. **密文-密文点积（余弦相似度）**：调用接口计算加密查询与候选姓名的余弦相似度。
7. **阈值减法**：调用接口从加密分数中减去设定阈值。
8. **随机掩码**：调用接口对减去阈值后的分数乘以随机正整数。
9. **解密与判断**：查询方解密最终分数，根据正负判断是否匹配。

六、上下游数据流梳理

1. 上游输入

本模块从以下两个上游模块接收数据：

上游模块	提供的数据	在本模块中的用途
加密模块	加密查询向量	作为点积运算的操作数，参与质心匹配、列选择和余弦相似度计算
聚类优化模块	明文聚类中心矩阵	作为密文-明文点积的明文操作数，用于计算查询与各聚类中心的相似度
聚类优化模块	明文列矩阵	作为密文-明文点积的明文操作数，用于从每一列中提取匹配聚类的姓名

2. 内部处理流程

本模块内部的数据流转逻辑如下：

1. 接收加密查询向量后，首先与明文聚类中心执行密文-明文点积，得到各聚类的加密相似度分数。

- 2. 将加密分数返回给查询方，不在本模块内解密。
- 3. 收到查询方返回的加密独热向量后，与列矩阵执行密文-明文点积，提取出匹配聚类的姓名向量。
- 4. 将提取出的姓名向量与原始加密查询向量执行密文-密文点积，得到加密的余弦相似度。
- 5. 对加密余弦相似度执行阈值减法，得到加密的差值。
- 6. 对加密差值执行随机掩码乘法，得到最终的加密分数。

3. 下游输出

本模块将以下数据传递给下游环节：

下游接收方	输出的数据	核心用途
查询方	每列的加密分数	查询方解密后，根据分数正负判断是否存在匹配的姓名
查询方	最终匹配结果	导出匹配标识值（0 表示无匹配，1 表示存在匹配）